Group C: Blockchain Technology (410243)

Unit-wise viva questions with short spoken-style answers, then questions on practicals 13–17.

Unit I: Mathematical Foundation for Blockchain

Must know: symmetric vs asymmetric cryptography, ECC, hash functions and SHA-256, digital signatures (DSA/ECDSA), Merkle trees.

	Symmetric	Asymmetric
Keys	One shared secret key	Public key + private key pair
Speed	Fast	Slow (heavy maths)
Key distribution	Hard (must share the secret safely)	Easy (public key can be published)
Use	Bulk data encryption	Key exchange, digital signatures, wallet addresses
Examples	AES, DES, 3DES, Blowfish	RSA, ECC, Diffie–Hellman, ElGamal

Q1. What is cryptography, and what goals does it serve? The science of securing information. Goals: confidentiality, integrity, authentication and non-repudiation.

Q2. Symmetric key cryptography? The same secret key encrypts and decrypts. It is fast, but both parties must share the key securely, and n users need $n(n-1)/2$ keys. AES is the standard.

Q3. Asymmetric (public key) cryptography? Each user has a key pair. Encrypting with the receiver's public key means only their private key can decrypt (confidentiality). Signing with your private key lets anyone verify with your public key (authenticity).

Q4. Which cryptography does blockchain use, and where? Asymmetric cryptography for wallets and signatures (ECDSA on the secp256k1 curve in Bitcoin and Ethereum), and hash functions for linking blocks, Merkle trees, addresses and Proof of Work.

Q5. What is Elliptic Curve Cryptography (ECC)? Public-key cryptography based on points on an elliptic curve $y^2 = x^3 + ax + b$ over a finite field. The private key is a number k; the public key is the point $K = k \cdot G$ (G is a fixed generator point). Finding k from K is the

elliptic curve discrete logarithm problem, which is infeasible.

Q6. Why ECC instead of RSA? Equal security with much smaller keys: a 256-bit ECC key ≈ a 3072-bit RSA key. That means faster computation, less storage and smaller signatures, which suits blockchains and mobile devices.

Q7. What is a cryptographic hash function? A function that maps input of any size to a fixed-size output (digest). Properties: deterministic, fast to compute, pre-image resistant (one-way), second pre-image resistant, collision resistant, and the avalanche effect (a one-bit change alters about half the output bits).

Q8. What is SHA-256? Secure Hash Algorithm 2 with a 256-bit (64 hex character) output, designed by the NSA. It processes 512-bit blocks through 64 rounds. Bitcoin uses double SHA-256 for block hashes and mining; Ethereum uses Keccak-256 instead.

Q9. How does hashing make a blockchain tamper-evident? Each block header stores the hash of the previous block. Changing any old transaction changes that block's hash, which breaks the link to every later block; an attacker would have to redo all subsequent blocks faster than the honest network.

Q10. What is a digital signature? The sender hashes the message and encrypts (signs) the hash with their private key; the receiver verifies it with the sender's public key. It provides authentication, integrity and non-repudiation. In blockchain, every transaction is signed by the owner of the coins.

Q11. What is DSA? The Digital Signature Algorithm, a NIST standard based on the discrete logarithm problem; the signature is a pair (r, s). ECDSA is its elliptic-curve version, which Bitcoin and Ethereum use.

Q12. What is a Merkle tree? A binary tree of hashes: leaves are hashes of transactions, each parent is the hash of its two children combined, and the top is the Merkle root, stored in the block header. Any change to any transaction changes the root.

Q13. Why are Merkle trees useful? They summarise thousands of transactions in one 32-byte root, and allow a Merkle proof: verifying that a transaction is in a block with only O(log n) hashes. Light clients (SPV wallets) use this without downloading full blocks.

Q14. Case study: compare symmetric and asymmetric algorithms. AES (symmetric, 128/256-bit keys) is very fast for bulk data. RSA (asymmetric, 2048+ bit keys) is slow but solves key distribution. ECC (asymmetric, 256-bit) gives RSA-level security with small keys. In practice they are combined (hybrid encryption, as in TLS): asymmetric to exchange a symmetric session key.

Unit II: Blockchain Basics

Must know: history, centralized vs decentralized vs distributed, the five layers, why blockchain matters, limits of centralized systems, adoption so far. (The printed syllabus mistakenly titles this unit “Feature Engineering”; its content is blockchain basics.)

Layer	Role	Examples
Application	Interfaces users interact with	Wallets, dApps, exchanges
Execution	Runs the code and rules	Smart contracts, chaincode, EVM
Semantic	Defines valid transactions and links blocks	Transaction rules, block structure, hash links
Propagation	Peer-to-peer communication	Gossip protocol, broadcasting transactions and blocks
Consensus	Agreement on the state of the ledger	PoW, PoS, PBFT, PoET

Q1. What is a blockchain? A distributed, decentralized, append-only ledger. Transactions are grouped into blocks, each block contains the hash of the previous one, and copies are kept by many nodes that agree on the contents through consensus.

Q2. What does a block contain? A header (previous block hash, Merkle root, timestamp, nonce, difficulty target, version) and a body (the list of transactions). The first block is the genesis block.

Q3. Brief history of blockchain? 1991: Haber and Stornetta's time-stamped hash chain. 2008: Satoshi Nakamoto's Bitcoin whitepaper. 2009: the Bitcoin genesis block. 2015: Ethereum adds smart contracts. 2015–16: Hyperledger (Linux Foundation) for enterprise. 2022: Ethereum moves to Proof of Stake (“the Merge”).

Q4. Centralized vs decentralized vs distributed? Centralized: one authority controls everything (a bank's database), so there is a single point of failure. Decentralized: control is spread over many independent nodes and no one owner exists. Distributed: data and processing are spread across locations (which can still be centrally owned). Blockchain is both decentralized and distributed.

Q5. Limitations of centralized systems? A single point of failure and attack, the need to trust the intermediary, censorship, possible data tampering by the owner, lack of transparency, intermediary fees and delays (cross-border payments), and privacy

breaches from central data stores.

Q6. Why is blockchain important? It allows trust between parties who don't know each other without an intermediary. It offers immutability, transparency and auditability, fault tolerance (no single point of failure), faster settlement, and programmable money through smart contracts.

Q7. Key properties of blockchain? Decentralization, immutability, transparency, security (cryptography), consensus, traceability and, in public chains, pseudonymity.

Q8. Explain the layers of blockchain. Application (dApps and wallets) → execution (smart contracts run) → semantic (transaction validity and block linking) → propagation (peer-to-peer gossip of transactions and blocks) → consensus (nodes agree on one version of the ledger).

Q9. What is a node? Full node vs light node? A computer that participates in the network. A full node stores the whole blockchain and validates every block. A light (SPV) node stores only block headers and relies on full nodes, using Merkle proofs.

Q10. What is immutability, and is it absolute? Once recorded and confirmed, data can't practically be changed. It's not absolute: an entity controlling more than 50% of the hash power (a 51% attack) could rewrite recent blocks, and contentious hard forks can change history by social agreement.

Q11. What is a fork? Soft vs hard fork? A split in the chain. A soft fork is a backward-compatible rule tightening (SegWit). A hard fork is a non-backward-compatible change that splits the network if not everyone upgrades (Bitcoin Cash, Ethereum Classic).

Q12. What is the double-spending problem? Spending the same digital coin twice. Blockchain prevents it with a single agreed transaction order through consensus and confirmations (usually 6 blocks in Bitcoin).

Q13. Limitations of blockchain? Low throughput (Bitcoin about 7 transactions/second), high energy use (PoW), storage growth, irreversibility of mistakes, privacy issues on public chains, regulatory uncertainty, smart contract bugs, and the scalability trilemma (security, decentralization, scalability).

Q14. Blockchain adoption so far? Cryptocurrencies and DeFi, cross-border payments (RippleNet), supply chain tracking (IBM Food Trust, Walmart), trade finance (Contour, we.trade), digital identity, government land records (pilots in India such as Andhra Pradesh and Telangana), healthcare data sharing, NFTs.

Q15. Case study: how do you study a research paper on blockchain? Identify the problem addressed, the blockchain type and consensus used, the architecture, how the paper evaluates results (throughput, latency, cost), the limitations, and future work. Bitcoin's whitepaper (Nakamoto, 2008) is a good choice.

Unit III: Blockchain Platforms and Consensus

Must know: public/private/consortium; Bitcoin, Ethereum, Hyperledger, IOTA, Corda/R3; consensus elements; PoW, Byzantine Generals problem, PoS, PoET, Proof of Activity, Proof of Burn.

Consensus	How a block producer is chosen	Energy	Used by
Proof of Work	First to solve a hash puzzle by brute force	Very high	Bitcoin, Litecoin
Proof of Stake	Chosen in proportion to coins staked (locked)	Very low	Ethereum (since 2022), Cardano
Proof of Elapsed Time	Random wait time inside trusted hardware (Intel SGX); shortest wait wins	Low	Hyperledger Sawtooth
Proof of Activity	PoW mines an empty block header, then PoS-selected stakeholders sign and complete it	Medium	Decred (hybrid)
Proof of Burn	Send coins to an unspendable address; more burned = more chance to mine	Low	Slimcoin, Counterparty
PBFT	Voting rounds among known validators; tolerates f faulty nodes out of 3f+1	Low	Hyperledger Fabric 3 (BFT ordering), Zilliqa

Q1. Public vs private vs consortium blockchain? Public (permissionless): anyone can join, read and validate (Bitcoin, Ethereum). Private (permissioned): one organisation controls who participates (internal enterprise ledgers). Consortium: a group of organisations

jointly governs it (banking consortia, R3 Corda, Hyperledger Fabric networks). Hybrids also exist.

Q2. Permissionless vs permissioned? Permissionless: no identity checks, anonymous participants, needs strong consensus like PoW/PoS. Permissioned: participants are known and vetted, so lighter, faster consensus (PBFT, Raft) and privacy controls are possible.

Q3. Bitcoin as a platform? The first cryptocurrency (2009), which uses PoW with SHA-256, a 10-minute block time, a limited scripting language (not Turing-complete), the UTXO model, and a fixed supply of 21 million BTC.

Q4. Ethereum as a platform? Launched in 2015 by Vitalik Buterin and others. A programmable blockchain with Turing-complete smart contracts running on the EVM, an account-based model, Ether (ETH) as its currency, gas fees, and Proof of Stake since September 2022.

Q5. What is Hyperledger? An open-source umbrella project of the Linux Foundation for enterprise permissioned blockchains (renamed LF Decentralized Trust in 2024). Projects include Fabric (modular, chaincode in Go/Java/JS, channels for privacy, no native cryptocurrency), Sawtooth (PoET), Besu (Ethereum client), Indy (identity), and the now-deprecated Composer.

Q6. What is IOTA? A distributed ledger for the Internet of Things that uses a DAG called the Tangle instead of blocks. Each new transaction approves two earlier ones; it has no miners and no transaction fees, and suits machine-to-machine micropayments.

Q7. What is Corda / R3? R3 is a company leading a consortium of financial institutions. Corda is its permissioned DLT for finance: data is shared only between the parties to a transaction (not broadcast to all), and notaries prevent double spending. Strictly, it's a distributed ledger rather than a chain of blocks.

Q8. What is consensus, and why is it needed? The mechanism by which distributed nodes, with no central authority and some possibly faulty or malicious, agree on a single valid order of transactions (the state of the ledger). It prevents double spending and forks.

Q9. Elements of a consensus approach? Block proposal (who creates the next block), information propagation, block validation, finality (when a block can't be reversed), and incentives or penalties (rewards, fees, slashing).

Q10. Explain the Byzantine Generals problem. Several generals surrounding a city must agree to attack or retreat by messenger, but some generals may be traitors sending conflicting messages. Agreement is possible only if fewer than one-third are traitors ($n \geq$

3f + 1). Blockchain consensus solves this problem for computers; Byzantine Fault Tolerance (BFT) means tolerating such faulty or malicious nodes.

Q11. How does Proof of Work work? Miners repeatedly change the nonce and hash the block header until the hash is below the difficulty target (starts with enough zeros). Finding it is hard; verifying it takes one hash. The winner adds the block and gets the block reward plus fees. Difficulty adjusts every 2016 blocks to keep 10-minute blocks.

Q12. Pros and cons of PoW? Pros: proven secure, fully permissionless, attacks are very expensive. Cons: huge energy consumption, slow throughput, mining centralised in large pools, and vulnerability to 51% attacks on small chains.

Q13. How does Proof of Stake work? Validators lock up (stake) coins; the protocol pseudo-randomly selects a block proposer weighted by stake, and others attest. Honest behaviour earns rewards; misbehaviour (double-signing) is punished by slashing part of the stake. Ethereum requires 32 ETH per validator.

Q14. PoW vs PoS? PoW secures the chain with computation and electricity; PoS secures it with locked capital. PoS uses about 99.9% less energy, has faster finality and doesn't need special hardware, but critics worry about rich validators gaining more control and the "nothing at stake" problem (solved by slashing).

Q15. What is Proof of Elapsed Time? Each node asks trusted hardware (an Intel SGX enclave) for a random wait time; the node whose timer expires first creates the block, and the hardware proves the node really waited. It's fair and energy-efficient but depends on trusting Intel. Used in Hyperledger Sawtooth.

Q16. What are Proof of Activity and Proof of Burn? Proof of Activity is a hybrid: miners use PoW to create a block template, then a random set of stakeholders (PoS) must sign it to complete the block. Proof of Burn: miners destroy coins by sending them to an unspendable address, a "virtual mining rig"; the more burned, the higher the chance of being chosen.

Q17. What is a 51% attack? If one entity controls more than half of the mining power (PoW) or stake (PoS), it can reverse its own recent transactions (double spend) and censor others. It can't steal other people's coins or forge signatures.

Unit IV: Cryptocurrency, Bitcoin and Tokens

Must know: what cryptocurrency is, how Bitcoin works, types of cryptocurrency, coins vs tokens, uses, and wallets (MetaMask, Coinbase, Binance); case study: creating your own wallet.

Q1. What is a cryptocurrency? A digital currency secured by cryptography and recorded on a blockchain, with no central bank or authority. Ownership is proven with private keys, and transfers are validated by the network.

Q2. How does a Bitcoin transaction work? Alice creates a transaction that sends coins to Bob's address and signs it with her private key. She broadcasts it to the network; it waits in the mempool; a miner includes it in a block; after the block is mined and more blocks follow (confirmations), the transfer is final.

Q3. What is the UTXO model? Unspent Transaction Output. Bitcoin doesn't store balances; a wallet's balance is the sum of the unspent outputs it can unlock. A transaction consumes whole UTXOs as inputs and creates new outputs (to the receiver, plus change back to the sender). Ethereum uses an account-balance model instead.

Q4. Key facts about Bitcoin? Created by the pseudonymous Satoshi Nakamoto (whitepaper 2008, launched January 2009). Supply is capped at 21 million BTC. The block reward halves every 210,000 blocks (about 4 years); it is 3.125 BTC since the April 2024 halving. The smallest unit is 1 satoshi = 10^{-8} BTC.

Q5. What is mining? The process by which miners validate transactions, bundle them into blocks and compete to solve the PoW puzzle. It secures the network and issues new coins (the block reward) plus transaction fees.

Q6. Types of cryptocurrency? Bitcoin; altcoins (Litecoin, Ethereum's ETH, Solana, Cardano); stablecoins pegged to fiat money (USDT, USDC); utility tokens (access to a service, e.g. LINK, BNB); security tokens (represent investments); governance tokens (voting, e.g. UNI); privacy coins (Monero, Zcash); CBDCs (central-bank digital currencies, e.g. India's e-Rupee pilot) are digital currencies but centralised.

Q7. Coin vs token? A coin is native to its own blockchain (BTC on Bitcoin, ETH on Ethereum). A token is created by a smart contract on an existing blockchain, e.g. ERC-20 tokens on Ethereum.

Q8. What are ERC-20 and ERC-721? Ethereum token standards. ERC-20 is for fungible tokens (every unit identical; functions such as transfer, balanceOf, approve). ERC-721 is for non-fungible tokens (NFTs), where each token is unique (digital art, collectibles). ERC-1155 supports both.

Q9. What is cryptocurrency used for? Peer-to-peer payments, cross-border remittances, store of value, trading and investment, DeFi (lending, borrowing, decentralized exchanges), paying gas fees, crowdfunding (ICOs), NFTs and gaming.

Q10. What is a crypto wallet? Software or hardware that stores your private keys (not the coins themselves, which live on the blockchain), generates addresses, signs transactions and shows balances.

Q11. Hot wallet vs cold wallet? Custodial vs non-custodial? Hot wallets are connected to the internet (MetaMask, mobile apps): convenient but riskier. Cold wallets are offline (Ledger and Trezor hardware, paper wallets): very secure. Custodial: a third party such as an exchange holds your keys. Non-custodial: you hold your keys ("not your keys, not your coins").

Q12. What is MetaMask? A non-custodial hot wallet available as a browser extension and mobile app for Ethereum and EVM-compatible chains (Polygon, BNB Chain). It lets websites (dApps) connect to your account and ask you to sign transactions, and it supports switching networks, including test networks like Sepolia.

Q13. What are Coinbase and Binance? Centralized exchanges (CEX) where users buy, sell and trade crypto with fiat money. Their exchange accounts are custodial. Each also offers a separate self-custody wallet (Coinbase Wallet; Binance Web3 Wallet / Trust Wallet).

Q14. What is a seed phrase (Secret Recovery Phrase)? 12 or 24 words (the BIP-39 standard) from which all of the wallet's private keys are derived. Anyone with it controls the funds. Never share it or store it online; losing it means losing access forever.

Q15. How is a wallet address created? Generate a random private key → derive the public key with ECC (secp256k1) → hash it. In Ethereum, the address is the last 20 bytes of the Keccak-256 hash of the public key, written as 0x followed by 40 hex characters.

Q16. Case study: create your own wallet. Install MetaMask → create a password → write down the 12-word recovery phrase offline → confirm it → the wallet shows your address. Switch to a test network (Sepolia), get free test ETH from a faucet, and send a transaction to another account; check it on Etherscan (Practicals 13 and 14).

Unit V: Ethereum Platform using Solidity

Must know: what Ethereum is, network types, EVM, smart contracts (purpose, types), writing and deploying with Solidity, Swarm, Whisper; case study: Truffle.

Q1. What is Ethereum? An open-source, decentralized blockchain platform with a built-in Turing-complete language for smart contracts and decentralized applications (dApps). Its native currency is Ether (ETH); 1 ETH = 10^{18} wei.

Q2. Bitcoin vs Ethereum? Bitcoin is mainly digital money: limited scripting, UTXO model, PoW, about 10-minute blocks. Ethereum is a world computer: smart contracts, account model, PoS since 2022, about 12-second blocks, and gas fees for computation.

Q3. Types of Ethereum networks? Mainnet (real ETH, real value); public testnets (Sepolia for apps and Hoodi for validators, free test ETH from faucets; Ropsten, Rinkeby, Goerli and Holesky are retired); local or private development networks (Ganache, Hardhat Network, Remix VM) running on your own machine.

Q4. What is the EVM? The Ethereum Virtual Machine: a sandboxed, stack-based virtual machine that every node runs to execute contract bytecode deterministically. Solidity compiles to EVM bytecode, and every opcode costs a fixed amount of gas.

Q5. EOA vs contract account? An Externally Owned Account is controlled by a private key (your MetaMask account) and can start transactions. A contract account is controlled by its code, has no private key, and runs only when called.

Q6. What is gas, and why does it exist? Gas measures computational work. Fee = gas used × gas price (in gwei; since EIP-1559 it is a base fee plus a priority tip). It pays validators and prevents infinite loops or spam: a transaction that runs out of gas reverts but still pays the fee.

Q7. What is a smart contract? A program stored at an address on the blockchain that executes automatically when its conditions are met, i.e. when a transaction calls it. It is transparent, deterministic, tamper-proof once deployed, and removes intermediaries. Nick Szabo coined the term in 1994.

Q8. Purpose and types of smart contracts? Purpose: automate agreements without trusted third parties. Types: financial (tokens, DeFi lending, escrow), legal (agreements that execute themselves), decentralized autonomous organisations (DAOs, voting), application logic (games, NFTs, supply chain), and multi-signature wallets.

Q9. What is Solidity? A statically typed, contract-oriented, high-level language for the EVM, influenced by JavaScript, C++ and Python. A file starts with an SPDX license line and `pragma solidity ^0.8.0;`.

Q10. Key Solidity concepts? State variables (stored permanently on-chain), functions, modifiers (onlyOwner), events (logs), mappings (key–value stores), structs, `msg.sender` (the caller), `msg.value` (wei sent), `require` / `revert` / `assert` (validation), and the constructor (runs once at deployment).

Q11. Function visibility and state mutability in Solidity? Visibility: public (anyone), external (only from outside), internal (this contract and children), private (this contract only). Mutability: view (reads state, no gas when called externally), pure (reads nothing), payable (can receive ETH).

Q12. Storage vs memory vs calldata? Storage is permanent on-chain state (expensive). Memory is temporary for the function call. Calldata is read-only function input (cheapest).

Q13. How do you deploy a smart contract? Write the .sol file → compile with solc to bytecode and ABI → send a deployment transaction from an account (through Remix, Truffle or Hardhat, with MetaMask or a local network) → the contract gets an address → interact with it using the ABI.

Q14. What is the ABI? The Application Binary Interface: a JSON description of a contract's functions, inputs and outputs that front-ends (web3.js, ethers.js) use to encode calls to it.

Q15. What is Swarm? Ethereum's decentralized storage and content-distribution platform: files are split into chunks, stored across peer nodes and addressed by content hash, with incentives through the BZZ token. It's used to host dApp front-ends and data that is too big to store on-chain. (IPFS is a similar alternative.)

Q16. What is Whisper? Ethereum's peer-to-peer decentralized messaging protocol, part of the original "web3" trio (Ethereum = compute, Swarm = storage, Whisper = messaging). It gives dApps private, low-bandwidth communication. It has since been superseded by Waku.

Q17. What is Truffle? A development framework for Ethereum: project scaffolding (contracts/, migrations/, test/), compilation, migration scripts for deployment, automated testing in JavaScript/Solidity, and the Truffle console. Ganache provides a local blockchain with 10 funded accounts. (ConsenSys has since retired Truffle; Hardhat and Foundry are now common.)

Q18. Common Truffle commands? `truffle init` (create a project), `truffle compile`, `truffle migrate` (deploy, with `--network sepolia` for a testnet), `truffle test`, `truffle console`.

Q19. What is a re-entrancy attack? A malicious contract calls back into the victim's withdraw function before the victim updates its balance, draining funds repeatedly (the 2016 DAO hack, about $60M). Prevent it by updating state before sending ETH (the Checks-Effects-Interactions pattern used in Practical 15) or with a re-entrancy guard.

Unit VI: Blockchain Case Studies

Must know: at least one concrete use case per domain, the problem blockchain solves there, and its limits. For each domain, answer in three parts: the problem, how blockchain helps, and a real example.

Domain	Problem solved	How blockchain helps	Examples
Retail / supply chain	Fake goods, slow recalls, opaque sourcing	Tamper-proof tracking of a product from farm or factory to shelf	IBM Food Trust (Walmart traces mangoes in 2.2 s instead of 7 days), VeChain, De Beers Tracr (diamonds)
Banking and finance	Slow, costly cross-border payments, reconciliation, KYC duplication	Shared ledger, near-instant settlement, smart-contract trade finance, shared KYC	RippleNet, JPMorgan's Kinexys (formerly Onyx), DeFi lending, tokenized bonds
Government	Land fraud, record tampering, voting trust, subsidy leakage	Immutable land registries, transparent tenders, digital identity, e-voting pilots	Estonia's e-governance (KSI), land-record pilots in Indian states, Dubai Blockchain Strategy
Healthcare	Fragmented patient records, counterfeit drugs, clinical-trial integrity	Patient-controlled shared records, drug traceability, audit trails	MedRec (MIT), MediLedger (drug supply chain)
IoT	Central servers are bottlenecks; device identity and trust	Device identity, secure data logs, machine-to-machine micropayments	IOTA Tangle, Helium network
Energy and utilities	Centralised grids, renewable certificate fraud	Peer-to-peer solar trading, renewable energy certificates	Power Ledger, Energy Web Chain

Q1. What are the most prominent blockchain applications? Cryptocurrencies and DeFi, supply-chain traceability, cross-border payments, digital identity, healthcare records, land registries, voting, NFTs and digital ownership, and IoT data.

Q2. How does blockchain help supply chains in retail? Every handover (farm → processor → distributor → store) is recorded and can't be altered, so a retailer can trace a contaminated product's origin in seconds, prove authenticity and check ethical sourcing.

Q3. How is blockchain used in banking? Cross-border payments without correspondent-bank chains, trade finance with smart-contract letters of credit, shared KYC across banks, faster securities settlement, fraud reduction and tokenization of assets.

Q4. Blockchain in government? Tamper-proof land and property records, birth and degree certificates, transparent public procurement, digital identity, tax records, and e-voting pilots. Estonia is the classic example.

Q5. Blockchain in healthcare? Patients control access to their records across hospitals, drugs are traced to stop counterfeits, clinical-trial data is timestamped to prevent manipulation, and insurance claims are automated. Sensitive data stays off-chain, with only hashes or pointers on-chain.

Q6. Blockchain with IoT? Devices get secure identities and record sensor data on a ledger that can't be tampered with; smart contracts trigger actions (a shipment's temperature sensor exceeds a limit → automatic penalty). Problems: device limits and blockchain throughput, which is why IOTA uses a DAG.

Q7. Blockchain in energy? Peer-to-peer energy trading (a household with solar panels sells surplus to neighbours automatically), tracking renewable energy certificates and carbon credits, and EV charging payments.

Q8. How does blockchain integrate with other technologies? Blockchain + AI: data you can trust for models, auditable AI decisions. Blockchain + IoT: trusted device data. Blockchain + cloud: Blockchain-as-a-Service (AWS Managed Blockchain, Azure). Oracles (Chainlink) bring real-world data on-chain.

Q9. What is an oracle? A service that feeds external, off-chain data (prices, weather, sports results) into smart contracts, which otherwise can't access anything outside the blockchain. Chainlink is the best-known decentralized oracle network.

Q10. When should you NOT use blockchain? When one trusted party already controls the data, when high speed or throughput is needed, when data must be deleted or edited (GDPR's right to be forgotten), or when there's no need for shared trust among multiple parties. A normal database is better then.

Q11. What is an NFT? A Non-Fungible Token (ERC-721): a unique token on a blockchain that proves ownership of a digital or physical item such as art, collectibles, game items or tickets.

Q12. What is DeFi? Decentralized Finance: financial services (lending, borrowing, trading, stablecoins, insurance) built from smart contracts on public blockchains without banks. Examples: Uniswap (exchange), Aave (lending), MakerDAO (DAI stablecoin).

Q13. Case study: how would you write a report on two use cases? For each: the domain and problem, the blockchain type (public or permissioned) and why, the platform and consensus, the participants and data flow, the smart contracts used, the benefits seen (cost, time, fraud reduction), the challenges (scalability, regulation, adoption) and future scope. Example pair: IBM Food Trust (Hyperledger Fabric, permissioned) and Ethereum DeFi (public, PoS).

Lab practicals 13–17: questions on your work

Practical 13: MetaMask installation and spending Ether per transaction

- **What is MetaMask, and which type of wallet is it?** A browser-extension and mobile wallet for Ethereum and EVM chains; non-custodial and hot.
- **What does MetaMask store?** Your private keys (encrypted with your password), derived from the Secret Recovery Phrase. The ETH itself lives on the blockchain.
- **How is the Ether spent per transaction calculated?** Fee = gas used × (base fee + priority fee). A plain ETH transfer uses 21,000 gas; at 20 gwei that is 21,000 × 20 = 420,000 gwei = 0.00042 ETH.
- **What is gwei?** 10^{-9} ETH (10^{9} wei), the unit used for gas prices.
- **What is the gas limit?** The maximum gas you allow a transaction to use. Unused gas is refunded; running out reverts the transaction, and the gas used so far is still paid.
- **Why does a contract call cost more than a transfer?** It executes EVM opcodes and writes to storage, and each costs gas.

Practical 14: Creating a wallet

- **Steps to create the wallet?** Install the extension → Create a new wallet → set a password → save the 12-word recovery phrase offline → confirm it → the account and its 0x address are shown.

- **Password vs recovery phrase?** The password only unlocks MetaMask on this device. The recovery phrase restores the wallet anywhere and is the real master key.
- **How do you get test ETH?** Switch to the Sepolia test network and request ETH from a faucet using your address.
- **How do you send ETH?** Send → paste the recipient's address → amount → review the gas fee → Confirm. Track it on sepolia.etherscan.io with the transaction hash.
- **Why use a testnet?** It behaves like mainnet but test ETH has no value, so mistakes cost nothing.

Practical 15: Bank smart contract (Solidity)

- **Explain your contract's functions.** `deposit()` is payable and adds `msg.value` to `balances[msg.sender]`. `withdraw(amount)` checks the balance, deducts it, then sends ETH back. `getBalance()` is a view returning the caller's balance. Events log every deposit and withdrawal.
- **Why a mapping?** It stores a separate balance per customer address; unset keys default to 0.
- **What is `msg.sender`? `msg.value`?** The address calling the function; the amount of wei sent with the call.
- **What does `require` do?** It validates a condition; if false, the whole transaction reverts (all changes undone) with the error message.
- **Why deduct the balance before sending ETH?** The Checks-Effects-Interactions pattern prevents re-entrancy attacks.
- **Why does `getBalance` cost no gas?** It is a view function called locally without a transaction; deposit and withdraw change state, so they need a transaction and gas.
- **Why does your code use 0.8 and not the manual's 0.4.19?** Remix's current compilers are 0.8.x. Version 0.8 has built-in overflow checks and newer syntax: `constructor()`, `payable(msg.sender)`, `call{value: …}`.
- **Remix VM vs Injected Provider?** Remix VM is an in-browser simulated chain with 10 fake accounts; Injected Provider uses MetaMask to deploy to a real testnet like Sepolia.
- **What happens if you withdraw more than your balance?** `require` fails and the transaction reverts with "Insufficient balance".

Practical 16: Survey report on blockchain types

- **Types of blockchain and an example of each?** Public (Bitcoin, Ethereum), private (an internal Hyperledger Fabric deployment), consortium (R3 Corda banking networks, IBM Food Trust participants), hybrid (XinFin XDC, Dragonchain).
- **Real-time use cases to cite?** Walmart food traceability, RippleNet payments, Estonia's e-governance, Power Ledger energy trading, MediLedger drug supply chain.
- **Which type suits a bank, and why?** Permissioned (private or consortium): participants are known (KYC), transactions are private, throughput is higher, and regulation is easier to satisfy.

Practical 17: Hyperledger business network

- **What is Hyperledger Composer?** A toolset (now deprecated) on top of Hyperledger Fabric for modelling business networks quickly: a model file (.cto), a script (logic.js), access control (permissions.acl), packaged as a .bna file.
- **Participant vs asset vs transaction vs event in your network?** Employee is the participant; Hardware is the asset; Allocate is the transaction that changes the hardware's owner; HardwareAllocated is the event emitted afterwards.
- **What does `-->` mean in the .cto file?** A relationship (a reference to another resource), like a foreign key. `o` declares a normal field.
- **What do the `@param` and `@transaction` annotations do?** They tell Composer which transaction type the JavaScript function processes.
- **What happens if the transaction function throws an error?** The whole transaction is rejected and rolled back; no state changes.
- **What is the ACL file for?** Rules defining which participant may perform which operations (CREATE/READ/UPDATE/DELETE) on which resources.
- **What is a .bna file?** A Business Network Archive: the zipped model, script, ACL and package.json, deployed to Playground or Fabric.
- **Hyperledger Fabric vs Ethereum?** Fabric is permissioned, has no native cryptocurrency, uses pluggable consensus (Raft ordering), writes chaincode in Go/Java/JS, and uses channels for private data. Ethereum is public and permissionless with ETH, PoS and Solidity contracts.
- **What replaced Composer?** Writing chaincode directly with the Fabric SDKs and contract APIs (Fabric 2.x).